Job Scout: agent catalogue

VERSION 0.16 · 23 OCTOBER 2026 · ONE ENTRY PER AI COMPONENT · REVIEWED AT EVERY SPRINT CLOSE

This catalogue lists every place where an AI model makes a judgement. Everything else in Job Scout is ordinary code. A step became an agent only for isolation, independence or on-demand use (decision D20 in DECISIONS). Terms are explained in the glossary.

Every agent run adds an entry to the ledger (`data\ledger\<date>.json`) with the agent's name, the model, a hash of its instructions, the settings version and its inputs. So for any output it is possible to say which model wrote it, following which version of which instructions.

Summary

AGENT	LEDGER NAME	MODEL	RUNS	INSTRUCTIONS	CHECKED BY
Advert reader	reader	qwen2.5:14b (local)	07:45 daily, PC	`src\reader\` (prompt in code)	`src\reader\check.ts` grounding; `validate.py jobspec`
Reply reader	reply-reader	qwen2.5:14b (local)	After the advert reader, PC	`src\reader\reply.ts`	Grounding in code; `validate.py replies`
Finder (part 1)	orchestrator	Claude	07:23 daily, cloud	`SCOUT.md`, steps 0 to 5	`validate.py urls`
Scorer (part 2)	orchestrator	Claude	08:10 daily, cloud	`SCOUT.md`, `rubric.md`	`validate.py scores`, `carryover`, `card`, `moves`, `docs`
Writer	writer	Claude	Within part 2, per shortlisted role and per rebuild (E15)	`PACK.md`	`build_pack.py`; `validate.py pack`, `disclosure`
Reviewer	reviewer	Claude, separate agent	Within part 2, one per pack	`REVIEW.md`; `.claude\agents\reviewer.md`	`validate.py claims`
Weekly summary	weekly	Claude	08:50 Mondays, cloud	`WEEKLY.md`	`weekly.py` (code builds the figures)

The procedures (`SCOUT.md`, `PACK.md`, `REVIEW.md`, `WEEKLY.md`, `EVAL.md`, `REDTEAM.md`) live in the vault, `Professional development\_context\job-scout\`. The checkers are in `tools\`. No agent can change a checker.

Advert reader

- **Purpose.** Copy the facts from a job advert: title, company, place, working pattern, pay, must-haves, nice-to-haves, responsibilities, reporting line.
- **Why an agent.** Isolation. Adverts are written by strangers and may carry hidden instructions. The reader holds nothing worth stealing and can do nothing.
- **Model and settings.** qwen2.5:14b in Ollama, temperature 0, answers only in a fixed JSON shape. The model's digest (version fingerprint) and the Ollama runtime version are recorded every morning; a change to either gives a warning (`src\reader\pin.ts`).
- **Inputs.** One advert at a time, fetched by code (public job-board APIs first, robots.txt respected, never LinkedIn), cleaned by `src\untrusted.ts`.
- **Outputs.** `data\runs\<date>\specs\<id>.json`.
- **Tools and permissions.** None. No internet, no files.
- **Guardrails.** Code refuses pages without enough advert text. Code keeps only requirements and pay that appear in the advert's own words (grounding). The reader can also flag suspected injection.

- **Evaluation.** `npm run eval:reader`: 21 of 22 on 21 October 2026, the same in each of three runs (H19), as graded by hand on 16 October. Bar: 9 in 10. `--repeat 3` measures run-to-run variation. The one miss is real: on a long advert the model stops before the last two requirements. Set in `_context\job-scout\eval\reader\`.
- **Known limits.** Pages built by scripts cannot be read; part 2 reads those in the cloud and marks them. Speed depends on the graphics card (about 11 seconds an advert; ten times slower on the processor). Below 15 tokens a second the reader stops after the first advert and leaves the rest to part 2 in the cloud (G9).
- **Owner.** Code (local model); the user approves any model change.

Reply reader

- **Purpose.** Read a reply to an application and say what it is (rejection, interview invitation, request, offer and so on), with dates, a short quoted request and any calendar invitation.
- **Why an agent.** Isolation, as for the advert reader.
- **Model and settings.** As the advert reader.
- **Inputs.** Replies found by code: from hiring systems, LinkedIn application updates and senders on the watch list. Senders are judged by address and DKIM signature, never by display name. Calendar files are read by code, not the model.
- **Outputs.** `replies\summary.json`: type, company, role, dates, a request of at most 200 characters, and warnings (for example "asks for money or bank details").
- **Tools and permissions.** None. It never replies, accepts or opens links.
- **Evaluation.** `npm run grade-replies`: the user grades at least ten real replies. Bar: 9 in 10 right and no rejection read as an invitation. **Not graded yet**, so every board move it causes is a suggestion.
- **Owner.** Code (local model); the user grades.

Finder (part 1)

- **Purpose.** Find where each role's full advert is published, and give provisional scores so the reader knows what to read first.
- **Why an agent.** Web search needs judgement to pick the employer's own listing.
- **Inputs.** The board's settings; role records from `candidates.json`.
- **Outputs.** `advert-urls.json`; a copy of the settings for the PC (`config\settings.json`).
- **Tools and permissions.** Web search; the file bridge to the two shared folders (list, copy in, copy out; no commands); reading the board.
- **Guardrails.** Checks the pause switch first. Its link list is checked by `validate.py urls`.
- **Owner.** Cloud agent.

Scorer (part 2)

- **Purpose.** Score each role against eight weighted criteria, pick the best five (plus up to two at chosen target employers), write the digest and Draft cards, and match replies to applications.
- **Inputs.** Settings; job facts from the reader (or the advert read in the cloud when the PC could not); `rubric.md`; `candidate-profile.md`; reply summaries.
- **Outputs.** `scores.json`, `digest.md`, `carryover.json`, Draft cards, reply patches and Inbox entries on the board, `keep.json`, tomorrow's watch list; from 0.15.0 the staleness and rebuild patches on Draft cards (`validate.py packpatch`) and the day's run log (`runs`, `validate.py runlog`).
- **Tools and permissions.** The file bridge; the board's database (its own Draft cards and the reply patches only); never the settings document.
- **Guardrails.** Every output passes `validate.py` before it is written. Reply moves are decided by `tools\replies.py` (fixed rules in code), not by the agent.
- **Evaluation.** `EVAL.md`: agreement with the user's own apply or pass labels. **No baseline yet** (H18): the first set was mostly scored from alerts.
- **Owner.** Cloud agent; the user sets the criteria.

Writer

- **Purpose.** Draft the CV Summary, three highlights and a covering letter in the user's voice for each shortlisted role, plus a fit-and-gap note and form answers.
- **Inputs.** The job facts, the user's career record and positioning rules, their previous CV and letter.
- **Outputs.** A pack folder: CV and letter (PDF and Markdown), `fit-and-gap.md`, `form-answers.md`, `ats.json`, `pack.json` (written last, with `builtFrom`, what it was built from: E16, including the advert's facts, from the PC's spec or, for an advert read in the cloud, `staleness.py facts`: E17). A rebuild (E15) writes a new folder beside the old one and never touches the old.
- **Guardrails.** `build_pack.py` edits copies of the user's own documents and fails the build if the CV is not two pages, the letter runs past one page, or a barred claim or em dash appears. The approved answer on AI use must appear word for word (`validate.py disclosure`).
- **Owner.** Cloud agent; the user signs off every pack.

Reviewer

- **Purpose.** Judge every sentence of the drafts against the user's career record, and judge the keywords the code marked missing.
- **Why an agent.** Independence. It did not write the drafts and never sees the writer's reasoning (D26).
- **Inputs.** Only the drafts, `sentences.json`, `ats.json`, the form answers, the approved AI answer and the career record.
- **Outputs.** One JSON answer, saved as `claims.json` after checking.
- **Tools and permissions.** Read only. Loaded from `.claude\agents\reviewer.md` in the private repository where the scheduled task can load it; otherwise it runs on its instructions alone and still writes nothing.
- **Guardrails.** For every claim it accepts it must quote the record; `validate.py claims` confirms each quote exists and that the pack did not change during review. Unsupported lines go back to the writer once, then to the user.
- **Evaluation.** Hand-built cases 09 to 16, each with one planted problem: 8 of 8, 17 October 2026.
- **Owner.** Cloud agent.

Weekly summary

- **Purpose.** Summarise the week and, once a month, propose at most three changes from the user's edits and their reasons for declining roles.
- **Inputs.** The week's digests, scores, ledger and board; pack differences (`pack_diff.py`).
- **Outputs.** `data\weekly\<year>-W<week>.md`; proposals in `learning-log.md`.
- **Guardrails.** Figures are computed by `weekly.py`, not by the agent. Proposals are applied only after the user says yes.
- **Owner.** Cloud agent; the user decides.

Changing an agent

Before changing a model, an agent's instructions or the scoring criteria: run the evaluations (`npm run eval:reader`, `EVAL.md`, and `npm run grade-replies` for the reply reader), record the result, and update this catalogue and the system card. See OPERATIONS.